```
pip install ucimlrepo
```

```
Collecting ucimlrepo
  Downloading ucimlrepo-0.0.7-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2.2.2)
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2026.1.4)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.0
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlr
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2025
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (20
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0
Downloading ucimlrepo-0.0.7-py3-none-any.whl (8.0 kB)
Installing collected packages: ucimlrepo
Successfully installed ucimlrepo-0.0.7
```

1.Loading Dataset

```
from ucimlrepo import fetch_ucirepo
import numpy as np
import pandas as pd

# fetch dataset
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)

# data (as pandas dataframes)
X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets

# metadata
print(breast_cancer_wisconsin_diagnostic.metadata)

# variable information
print(breast_cancer_wisconsin_diagnostic.variables)
```

```
9      concave_points1  Feature   Continuous        None        None  None
10           symmetry1  Feature   Continuous        None        None  None
11  fractal_dimension1  Feature   Continuous        None        None  None
12             radius2  Feature   Continuous        None        None  None
13            texture2  Feature   Continuous        None        None  None
14          perimeter2  Feature   Continuous        None        None  None
15               area2  Feature   Continuous        None        None  None
16         smoothness2  Feature   Continuous        None        None  None
17        compactness2  Feature   Continuous        None        None  None
18          concavity2  Feature   Continuous        None        None  None
19     concave_points2  Feature   Continuous        None        None  None
20           symmetry2  Feature   Continuous        None        None  None
21  fractal_dimension2  Feature   Continuous        None        None  None
22             radius3  Feature   Continuous        None        None  None
23            texture3  Feature   Continuous        None        None  None
24          perimeter3  Feature   Continuous        None        None  None
25               area3  Feature   Continuous        None        None  None
26         smoothness3  Feature   Continuous        None        None  None
27        compactness3  Feature   Continuous        None        None  None
28          concavity3  Feature   Continuous        None        None  None
29     concave_points3  Feature   Continuous        None        None  None
30           symmetry3  Feature   Continuous        None        None  None
31  fractal_dimension3  Feature   Continuous        None        None  None

   missing_values
0              no
1              no
2              no
3              no
4              no
5              no
6              no
7              no
8              no
9              no
10             no
```

```
22              no
23              no
24              no
25              no
26              no
27              no
28              no
29              no
30              no
31              no
```

1.1. Encoding class labels

```
#Encode class labels
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y_encoded = le.fit_transform(y.values.ravel())
print(np.unique(y_encoded))
```

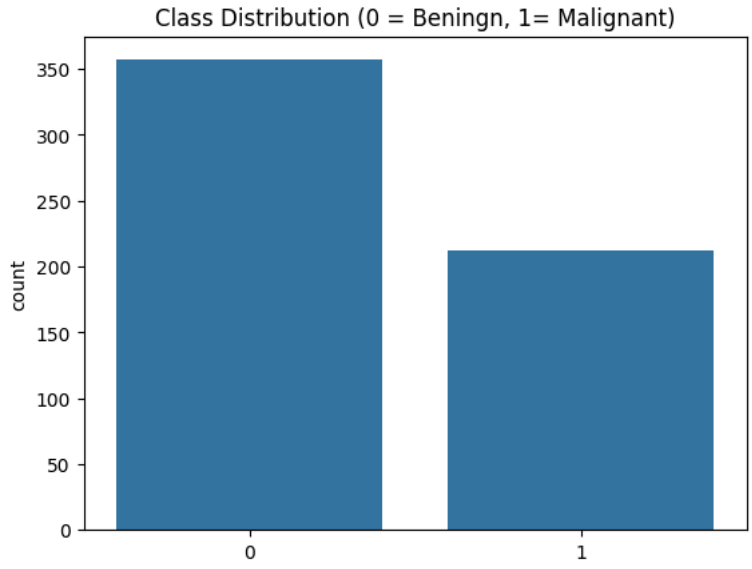
```
[0 1]
```

2. Exploratory Data Analysis

2.1. Class Distribution

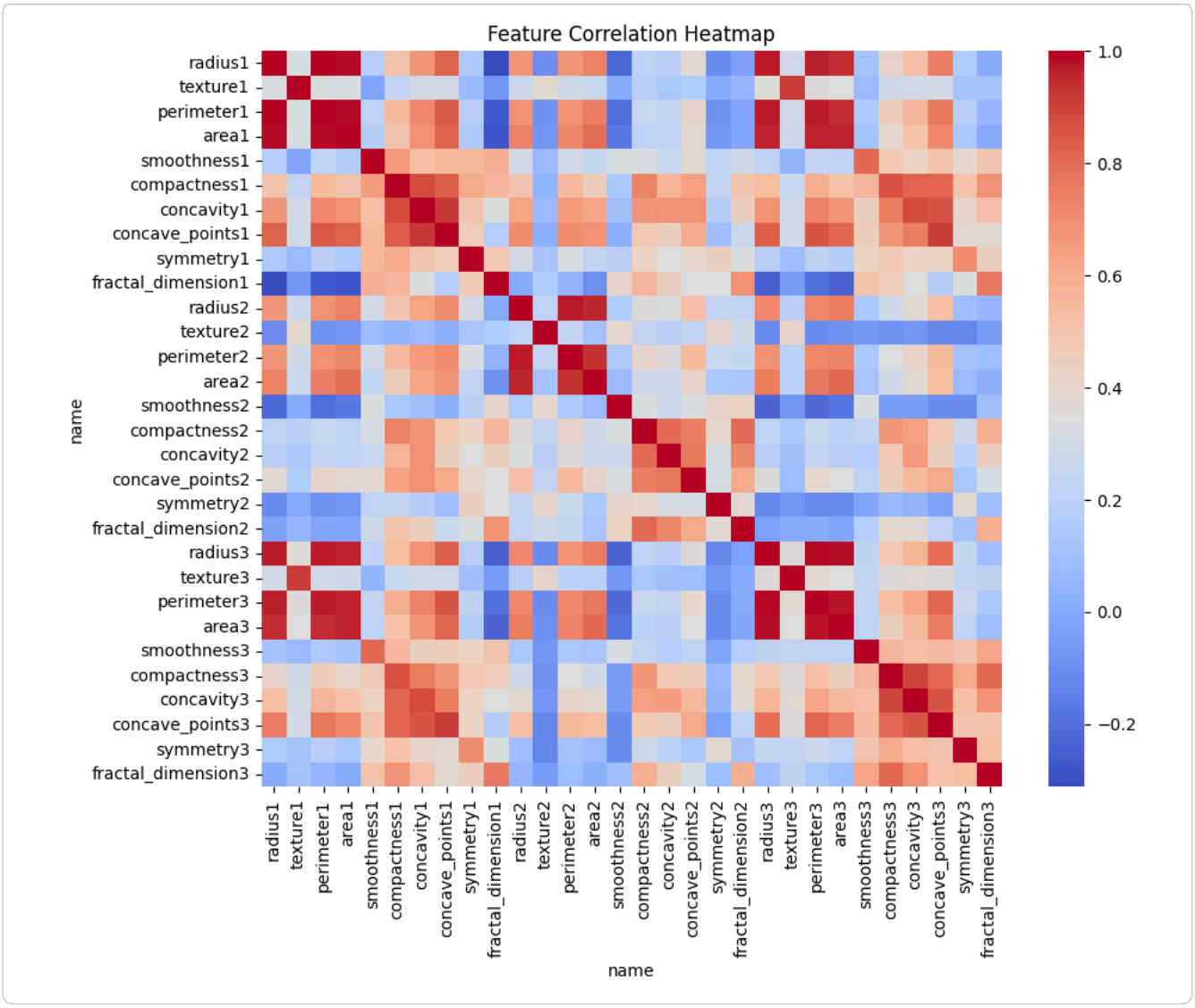
```
#visualization of class labels
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

y_series = pd.Series(y_encoded)
print(y_series.value_counts())

sns.countplot(x=y_encoded)
plt.title("Class Distribution (0 = Beningn, 1= Malignant)")
plt.savefig("Class comparision.png")
plt.show()
```

```
0    357
1    212
Name: count, dtype: int64
```



2.2 Feature correlation analysis

```
#Feature Correlation Analysis
X_df = pd.DataFrame(X, columns=breast_cancer_wisconsin_diagnostic.variables['name'][2:])
corr = X_df.corr()
plt.figure(figsize=(10,8))
sns.heatmap(corr, cmap="coolwarm")
plt.title("Feature Correlation Heatmap")
plt.savefig("Feature Correlation Heatmap.png")
plt.show()
```



3. Splitting dataset

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (455, 30)
Test shape: (114, 30)
```

4. Training a Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Train Decision Tree
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

# Predictions
y_pred = dt.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)
print("Decision Tree Accuracy:", acc*100)

# Detailed metrics
print(classification_report(y_test, y_pred))
```

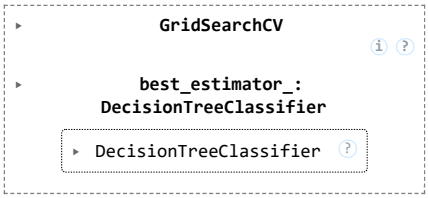
```
Decision Tree Accuracy: 92.98245614035088
              precision    recall  f1-score   support

           0       0.94      0.94      0.94        72
           1       0.90      0.90      0.90        42

    accuracy                           0.93       114
   macro avg       0.92      0.92      0.92       114
weighted avg       0.93      0.93      0.93       114
```

5. Define a Hyperparameter Search Space

```
param_grid = {
    "max_depth":[None,5,10,20,30],
    "min_samples_split":[2,5,10],
    "min_samples_leaf":[1,2,4],
    "criterion":["gini","entropy"],
    "max_features":[None,"sqrt","log2"]
}
```

6. 5-Fold Cross-Validation

```
from sklearn.model_selection import GridSearchCV
dt = DecisionTreeClassifier(random_state = 42)
grid_search = GridSearchCV(dt,param_grid,cv=5,scoring=["accuracy","f1"],refit="accuracy",n_jobs = -1)
grid_search.fit(X_train,y_train)
```




```
results = grid_search.cv_results_

table1 = pd.DataFrame({
    "Criterion":[p['criterion'] for p in results['params']],
    "Max Depth":[p['max_depth'] for p in results['params']],
    "Avg CV Accuracy (%)":results['mean_test_accuracy']*100,
    "Avg CV F1 Score":results['mean_test_f1']
})
table1_sorted = table1.sort_values(by="Avg CV Accuracy (%)",ascending=False)
print(table1_sorted.head(20))
```

```
    Criterion  Max Depth  Avg CV Accuracy (%)  Avg CV F1 Score
220   entropy       20.0            94.725275         0.929724
193   entropy       10.0            94.725275         0.929724
247   entropy       30.0            94.725275         0.929724
139   entropy        NaN            94.725275         0.929724
190   entropy       10.0            94.505495         0.927352
217   entropy       20.0            94.505495         0.927352
136   entropy        NaN            94.505495         0.927352
244   entropy       30.0            94.505495         0.927352
221   entropy       20.0            93.846154         0.918426
219   entropy       20.0            93.846154         0.918346
55       gini       10.0            93.846154         0.916672
1        gini        NaN            93.846154         0.916672
194   entropy       10.0            93.846154         0.918426
192   entropy       10.0            93.846154         0.918346
138   entropy        NaN            93.846154         0.918346
140   entropy        NaN            93.846154         0.918426
166   entropy        5.0            93.846154         0.917553
46       gini        5.0            93.846154         0.914934
109      gini       30.0            93.846154         0.916672
82       gini       20.0            93.846154         0.916672
```

7. Choosing Best Hyperparameters

```
print("Best Parameters:",grid_search.best_params_)
print("Best Accuracy:",grid_search.best_score_)
```

```
Best Parameters: {'criterion': 'entropy', 'max_depth': None, 'max_features': None, 'min_samples_leaf': 2, 'min_samples_split
Best Accuracy: 0.9472527472527472
```

```
print("Best F1 Score:",
      grid_search.cv_results_['mean_test_f1'][grid_search.best_index_])
```

```
Best F1 Score: 0.9297237101436056
```

```
import pandas as pd

results = pd.DataFrame(grid_search.cv_results_)
print(results[['params', 'mean_test_accuracy', 'mean_test_f1']])
```

```
                                                params  mean_test_accuracy  \
0    {'criterion': 'gini', 'max_depth': None, 'max_...            0.931868
1    {'criterion': 'gini', 'max_depth': None, 'max_...            0.938462
2    {'criterion': 'gini', 'max_depth': None, 'max_...            0.931868
3    {'criterion': 'gini', 'max_depth': None, 'max_...            0.931868
4    {'criterion': 'gini', 'max_depth': None, 'max_...            0.927473
..                                                 ...                 ...
265  {'criterion': 'entropy', 'max_depth': 30, 'max...            0.909890
266  {'criterion': 'entropy', 'max_depth': 30, 'max...            0.929670
267  {'criterion': 'entropy', 'max_depth': 30, 'max...            0.918681
268  {'criterion': 'entropy', 'max_depth': 30, 'max...            0.918681
269  {'criterion': 'entropy', 'max_depth': 30, 'max...            0.927473

     mean_test_f1
0        0.908902
1        0.916672
2        0.906730
3        0.906882
4        0.900641
..            ...
265      0.877700
266      0.905795
267      0.890332
268      0.890332
269      0.902305

[270 rows x 3 columns]
```

```
#Train the model
best_dt = grid_search.best_estimator_
y_pred_dt = best_dt.predict(X_test)

print("Tuned DT Accuracy:",accuracy_score(y_test,y_pred_dt)*100)
```

```
Tuned DT Accuracy: 95.6140350877193
```

8. Train a Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state = 42)
rf.fit(X_train,y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
y_pred_rf = rf.predict(X_test)
```

9. Hyperparameter Tuning for Random Forest

```
param_grid_rf = {
    "n_estimators":[50,100,200],
    "max_depth":[None,10,20],
    "min_samples_split":[2,5],
    "min_samples_leaf":[1,2],
    "max_features":["sqrt","log2"]
}

grid_search_rf = GridSearchCV(estimator = rf,param_grid = param_grid_rf,cv=5,scoring=["accuracy","f1"],refit="accuracy",n_jo
grid_search_rf.fit(X_train,y_train)
print("Best RF Parameters:",grid_search_rf.best_params_)
```

```
print("Best RF CV Accuracy:",grid_search_rf.best_score_)
```

```
Best RF Parameters: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators
Best RF CV Accuracy: 0.9670329670329672
```

```
results_rf = pd.DataFrame(grid_search_rf.cv_results_)

table_rf = results_rf[[
    "param_n_estimators",
    "param_max_depth",
    "param_max_features",
    "mean_test_accuracy",
    "mean_test_f1",
]].copy() # Explicitly create a copy to avoid SettingWithCopyWarning

table_rf["mean_test_accuracy"] *= 100

table_rf.columns = [
    "n_estimators",
    "Max Depth",
    "Max Features",
    "Avg CV Accuracy (%)",
    "Avg CV F1 Score",
]

print("Random Forest 5-Fold CV Performance Comparision")
print(table_rf.sort_values(by="Avg CV Accuracy (%)", ascending=False).head(20))
```

```
Random Forest 5-Fold CV Performance Comparision
    n_estimators Max Depth Max Features  Avg CV Accuracy (%)  Avg CV F1 Score
0             50      None         sqrt            96.703297         0.956130
24            50        10         sqrt            96.703297         0.956130
48            50        20         sqrt            96.703297         0.956130
1            100      None         sqrt            96.263736         0.950094
15            50      None         log2            96.263736         0.949585
25           100        10         sqrt            96.263736         0.950094
3             50      None         sqrt            96.263736         0.950094
7            100      None         sqrt            96.263736         0.950094
51            50        20         sqrt            96.263736         0.950094
49           100        20         sqrt            96.263736         0.950094
63            50        20         log2            96.263736         0.949585
27            50        10         sqrt            96.263736         0.950094
31           100        10         sqrt            96.263736         0.950094
39            50        10         log2            96.263736         0.949585
55           100        20         sqrt            96.263736         0.950094
6             50      None         sqrt            96.043956         0.946737
60            50        20         log2            96.043956         0.946027
57            50        20         sqrt            96.043956         0.946737
54            50        20         sqrt            96.043956         0.946737
30            50        10         sqrt            96.043956         0.946737
```

```
print(results_rf.columns)
```

```
Index(['mean_fit_time', 'std_fit_time', 'mean_score_time', 'std_score_time',
       'param_max_depth', 'param_max_features', 'param_min_samples_leaf',
       'param_min_samples_split', 'param_n_estimators', 'params',
       'split0_test_accuracy', 'split1_test_accuracy', 'split2_test_accuracy',
       'split3_test_accuracy', 'split4_test_accuracy', 'mean_test_accuracy',
       'std_test_accuracy', 'rank_test_accuracy', 'split0_test_f1',
       'split1_test_f1', 'split2_test_f1', 'split3_test_f1', 'split4_test_f1',
       'mean_test_f1', 'std_test_f1', 'rank_test_f1'],
      dtype='object')
```

```
#Train best model
best_rf = grid_search_rf.best_estimator_
y_pred_rf = best_rf.predict(X_test)
```

10. Compare Both Models

```
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score

def evaluate(y_test,y_pred,model_name):
  print(f"\n{model_name} Performance")
  print("Accuracy:",accuracy_score(y_test,y_pred))
  print("Precision:",precision_score(y_test,y_pred))
  print("Recall:",recall_score(y_test,y_pred))
  print("F1-Score:",f1_score(y_test,y_pred))

evaluate(y_test,y_pred_dt,"Decision Tree")
evaluate(y_test,y_pred_rf,"Random Forest")
```

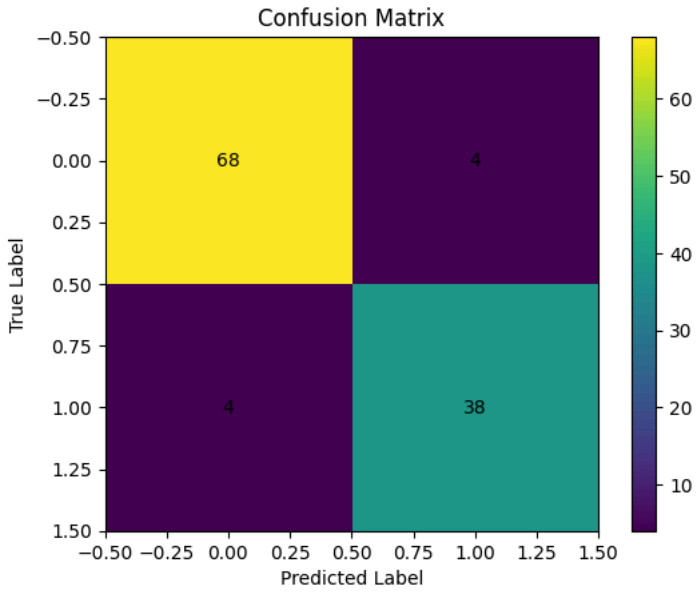
```
Decision Tree Performance
Accuracy: 0.956140350877193
Precision: 1.0
Recall: 0.8809523809523809
F1-Score: 0.9367088607594937

Random Forest Performance
Accuracy: 0.9736842105263158
Precision: 1.0
Recall: 0.9285714285714286
F1-Score: 0.9629629629629629
```

```
#5-Fold CV Performance Comparision
from sklearn.model_selection import cross_val_score

# 5-fold accuracy scores
dt_scores = cross_val_score(dt, X_train, y_train, cv=5, scoring="accuracy")
rf_scores = cross_val_score(rf, X_train, y_train, cv=5, scoring="accuracy")
```

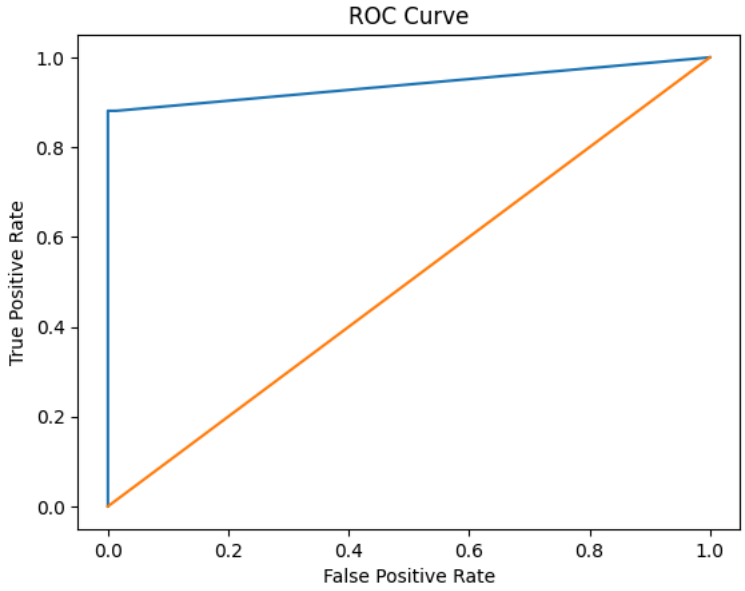
```
table = pd.DataFrame({
    "Model": ["Decision Tree", "Random Forest"],
    "Fold 1": [dt_scores[0]*100, rf_scores[0]*100],
    "Fold 2": [dt_scores[1]*100, rf_scores[1]*100],
    "Fold 3": [dt_scores[2]*100, rf_scores[2]*100],
    "Fold 4": [dt_scores[3]*100, rf_scores[3]*100],
    "Fold 5": [dt_scores[4]*100, rf_scores[4]*100],
    "Average": [np.mean(dt_scores)*100, np.mean(rf_scores)*100]
})

print(table)
```

```
           Model      Fold 1     Fold 2     Fold 3     Fold 4     Fold 5  \
0  Decision Tree   96.703297  95.604396  91.208791  93.406593  89.010989
1  Random Forest  100.000000  98.901099  93.406593  97.802198  91.208791

     Average
0  93.186813
1  96.263736
```

```
#Confusion Matrix

from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

cm = confusion_matrix(y_test,y_pred)
plt.figure()
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.colorbar()

for i in range(len(cm)):
  for j in range(len(cm)):
    plt.text(j,i,cm[i,j],ha="center",va="center")

plt.savefig("Confusion Matrix.png")
plt.show()
```




```
#ROC Curve (For Decision Tree)

from sklearn.metrics import roc_curve,auc
y_probs = best_dt.predict_proba(X_test)[:,1]
fpr,tpr,thresholds = roc_curve(y_test,y_probs)
roc_auc = auc(fpr,tpr)

plt.figure()
plt.plot(fpr,tpr)
plt.plot([0,1],[0,1])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")

plt.savefig("ROC Curve.png")
plt.show()
```

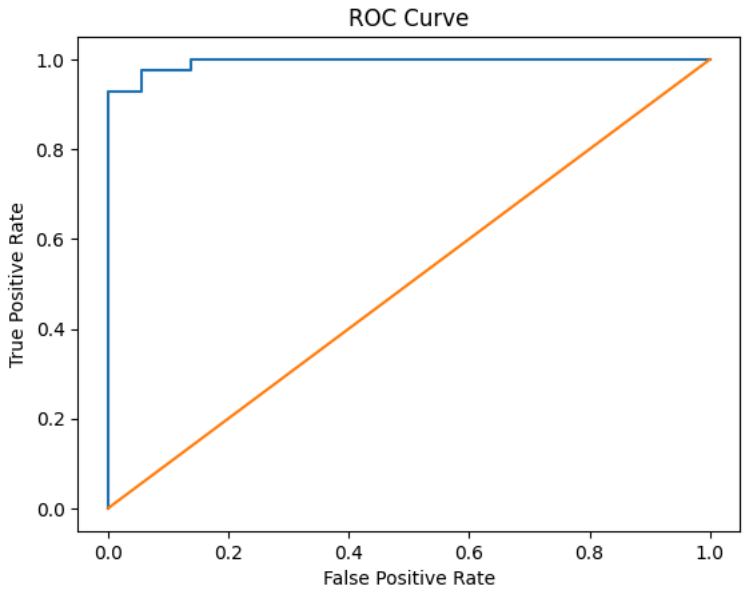



```
#AUC

print("AUC Score for decision tree:",roc_auc)
```

```
AUC Score for decision tree: 0.9396494708994709
```

```
#ROC Curve (for Random Forest)
from sklearn.metrics import roc_curve,auc


y_probs = best_rf.predict_proba(X_test)[:,1]
```

```
fpr,tpr,thresholds = roc_curve(y_test,y_probs)
roc_auc = auc(fpr,tpr)

plt.figure()
plt.plot(fpr,tpr)
plt.plot([0,1],[0,1])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")

plt.savefig("ROC Curve Random.png")
plt.show()
```




```
#AUC
print("AUC Score for decision tree:",roc_auc)
```

```
AUC Score for decision tree: 0.9940476190476191
```

Start coding or generate with AI.